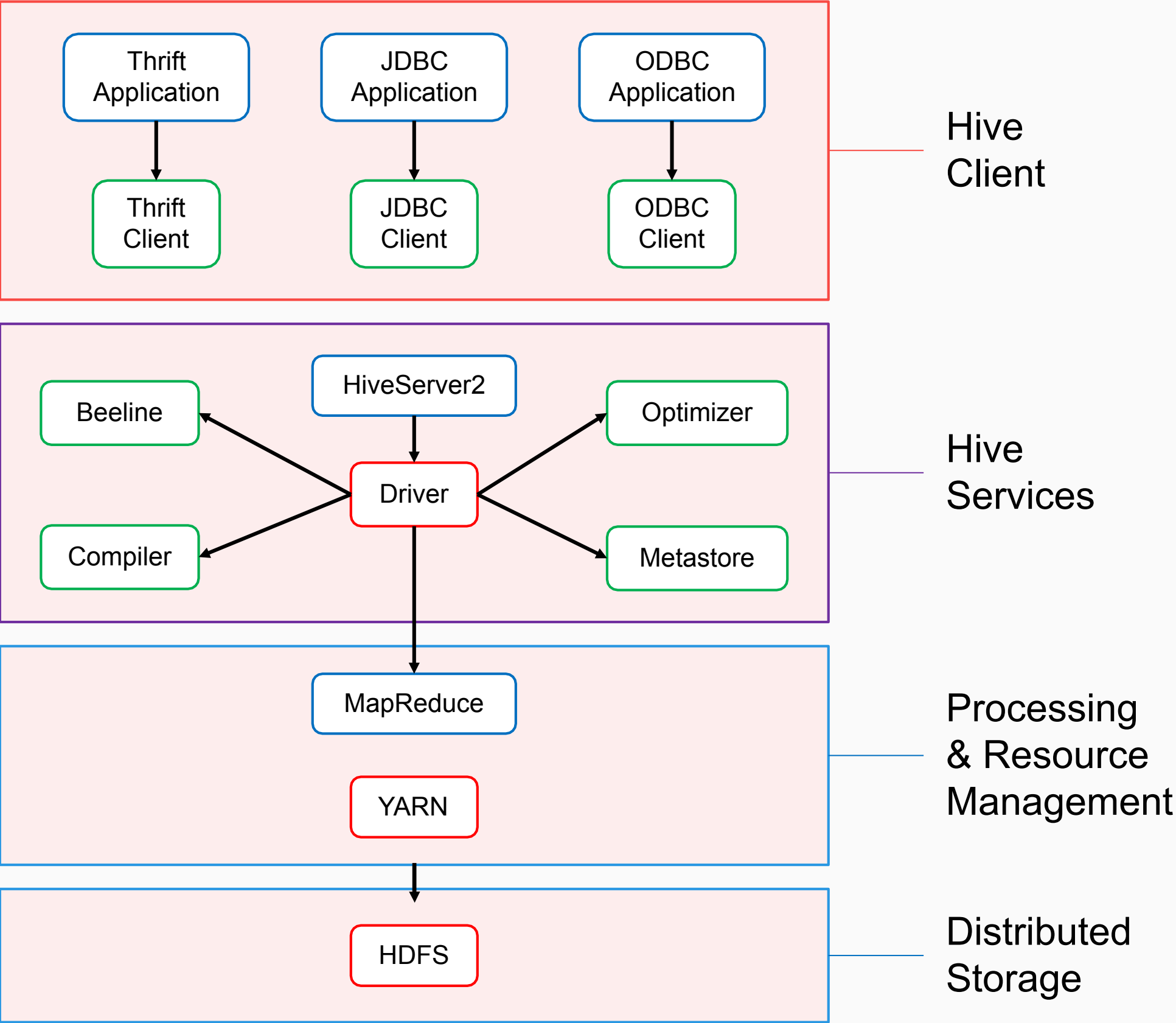


# Архитектура Hive

Skillbox

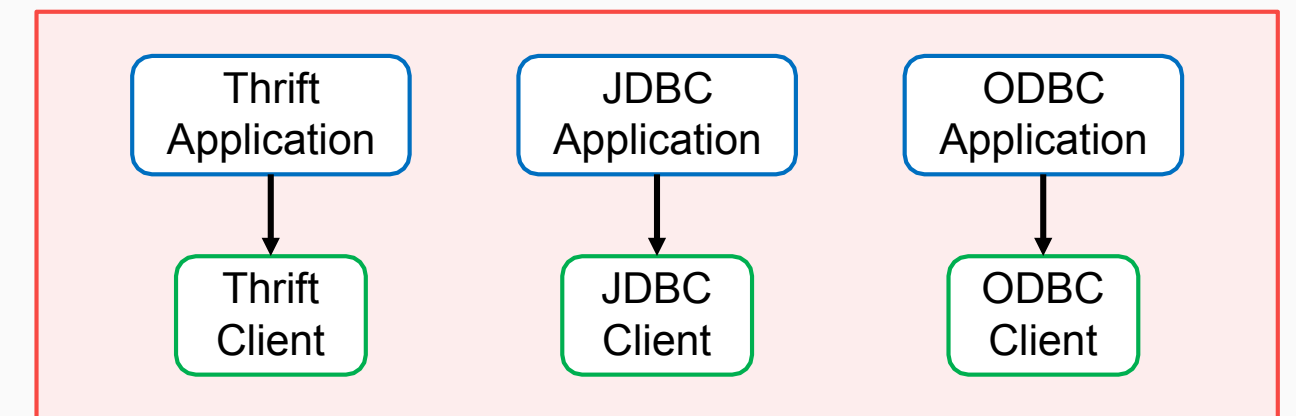
# Цели видео

- 1 Понять архитектуру Hive
- 2 Понять, что происходит после выполнения запроса
- 3 Разобраться с тем, что такое Execution-движок



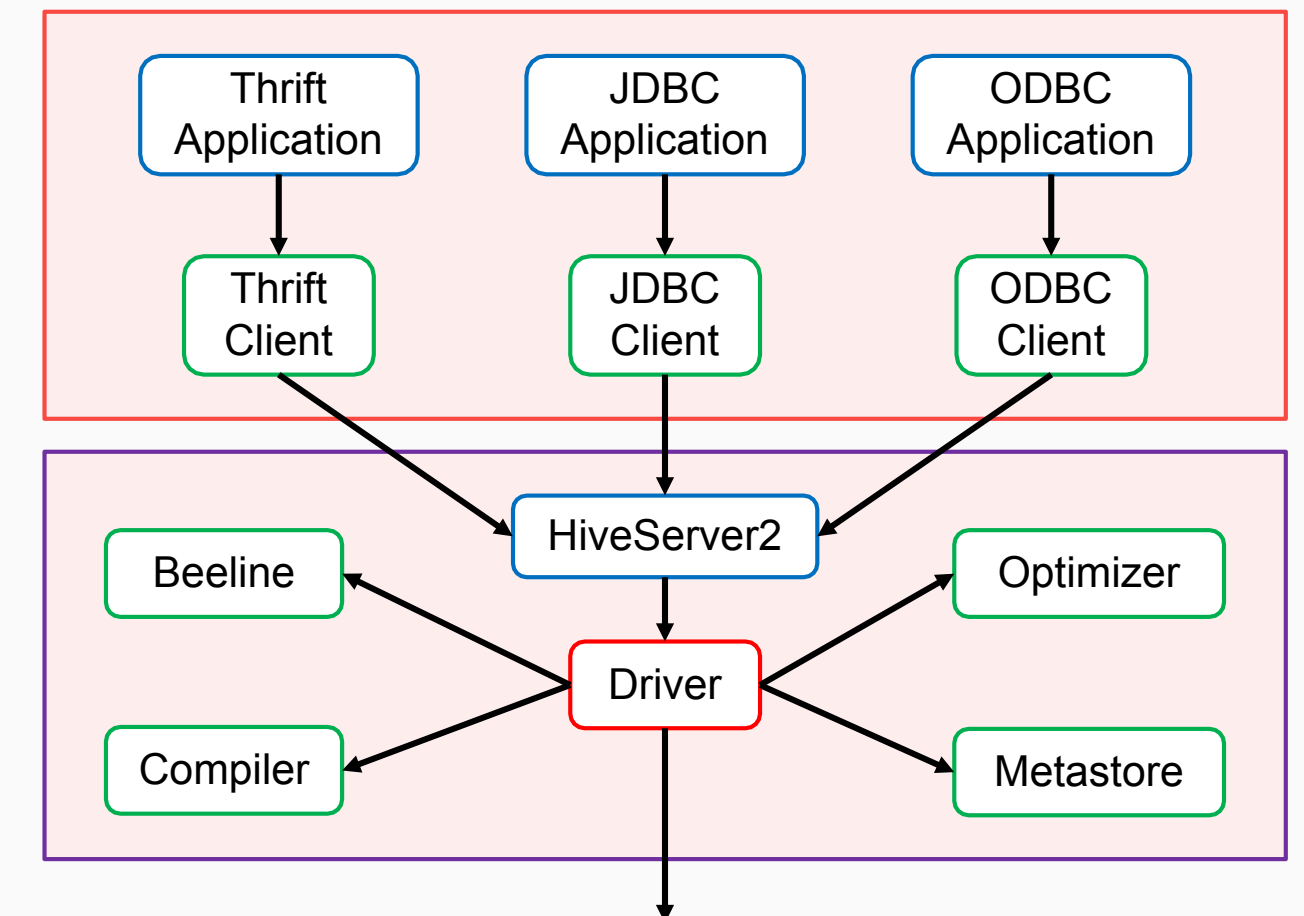
# Hive Client

- 1 Hive написан на Java
- 2 Под капотом Thrift  
Запросы от Thrift App
- 3
- 4 Запросы от JDBC App
- 5 Запросы от ODBC App



# Hive Services

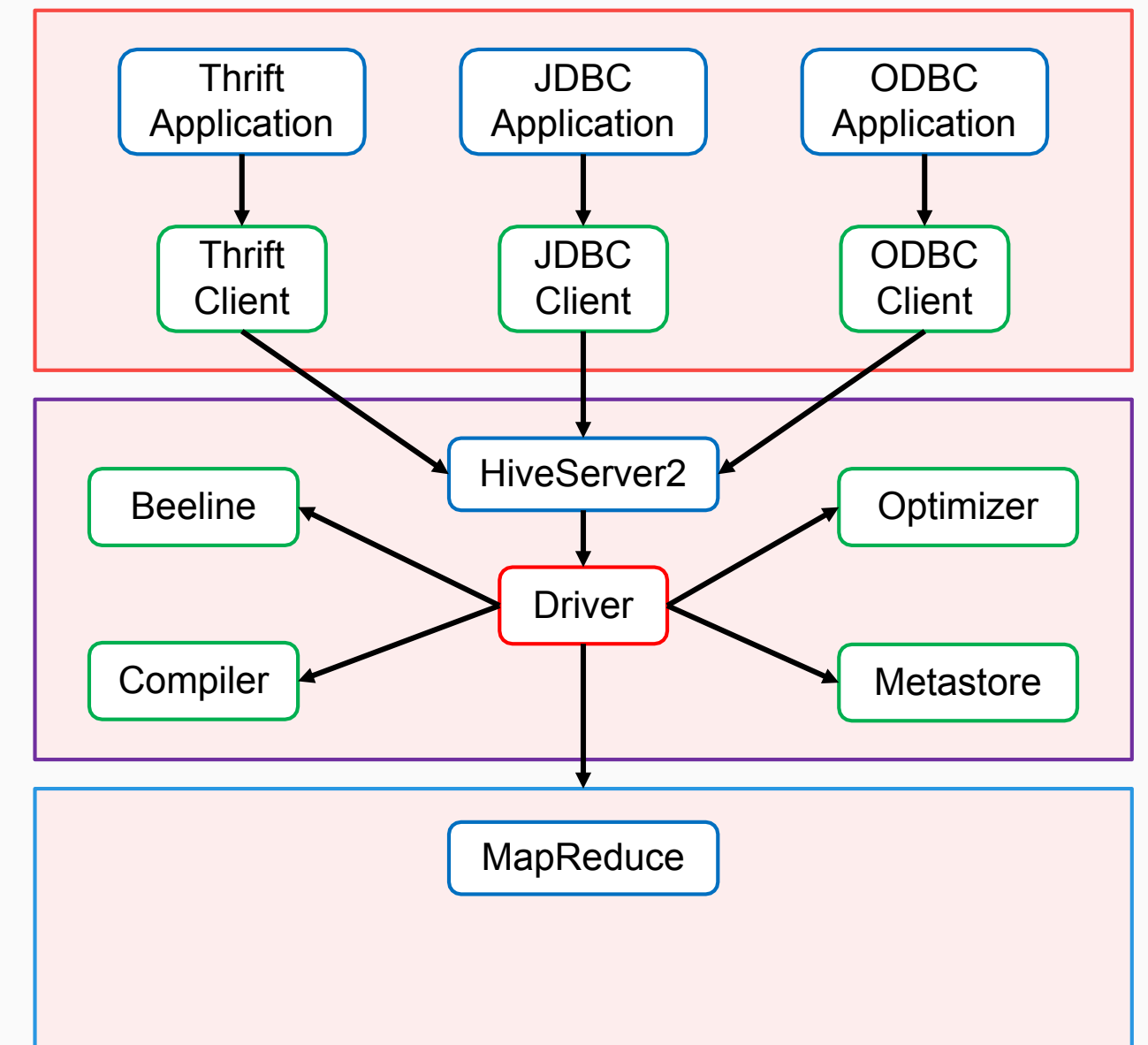
- 1 Beeline — консоль запросов
- 2 HiveServer2 — выполнение
- 3 Driver — координация
- 4 Compiler — генерация плана выполнения и DAG  
Optimizer — объединение, порядок тасок...
- 5 Metastore — хранение меты таблиц
- 6





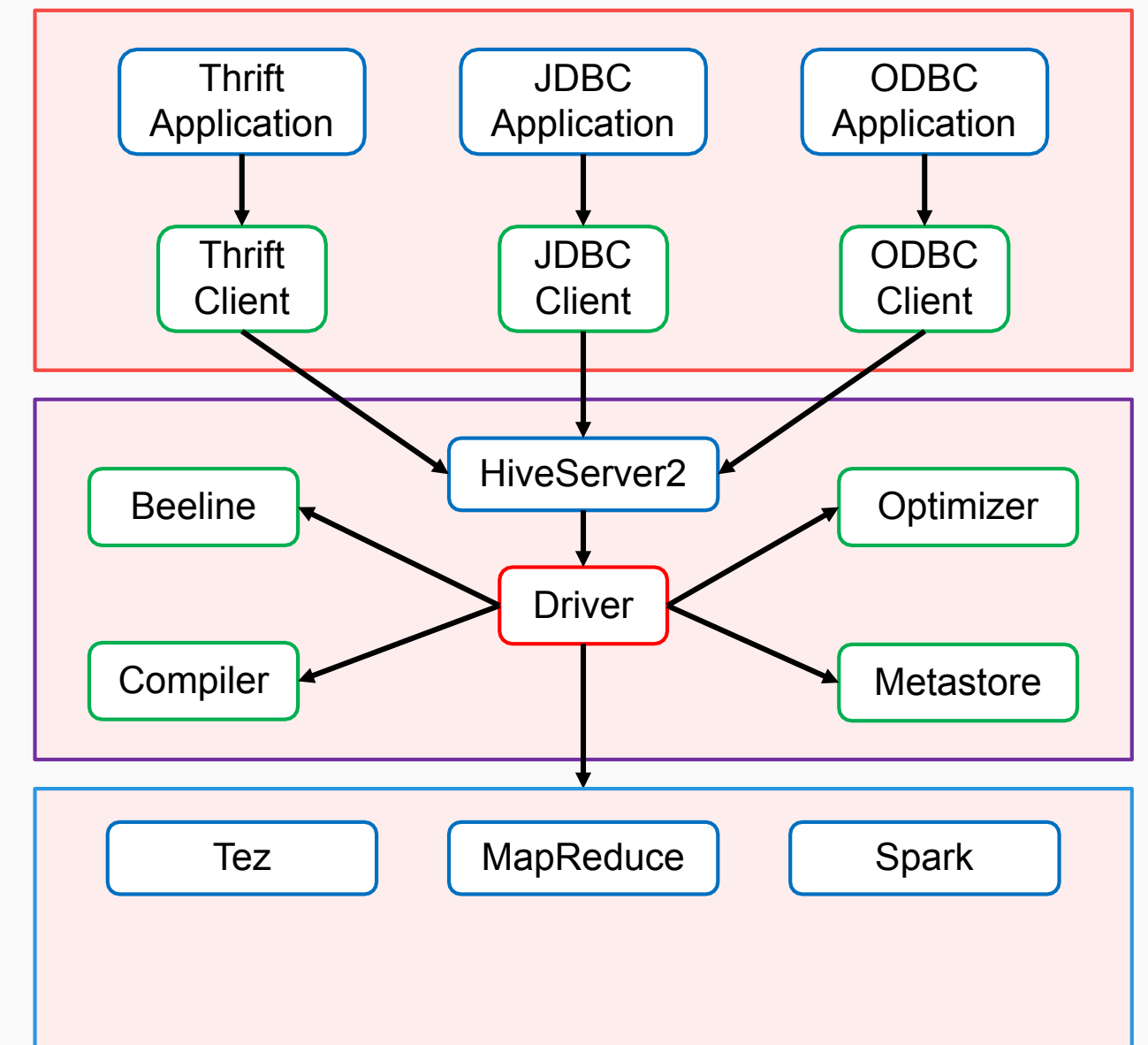
# Processing

- 1 Выполнение плана
- 2 MapReduce



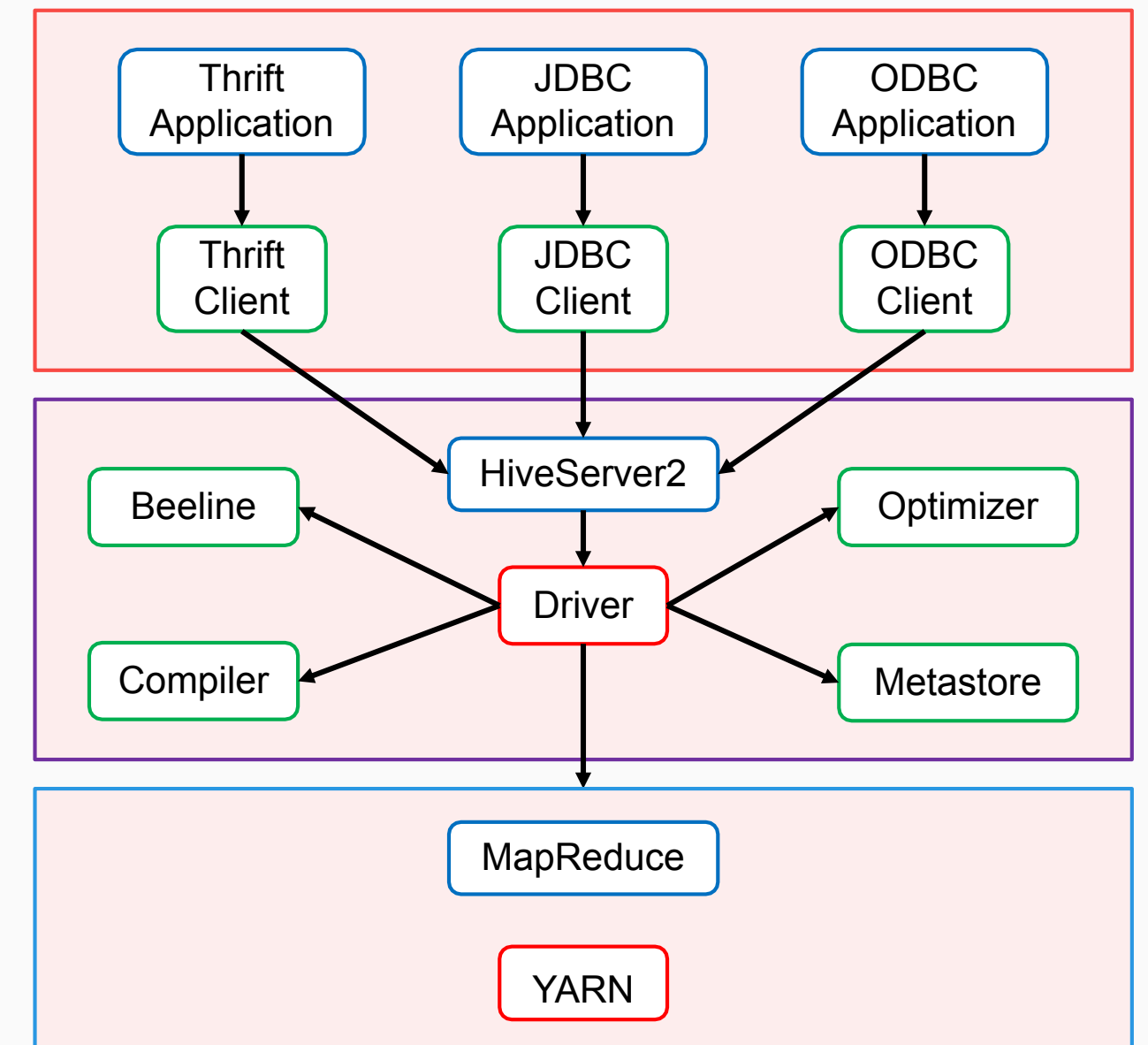
# Processing

- 1 Выполнение плана
- 2 MapReduce
- 3 Tez
- 4 Spark



# Управление ресурсами

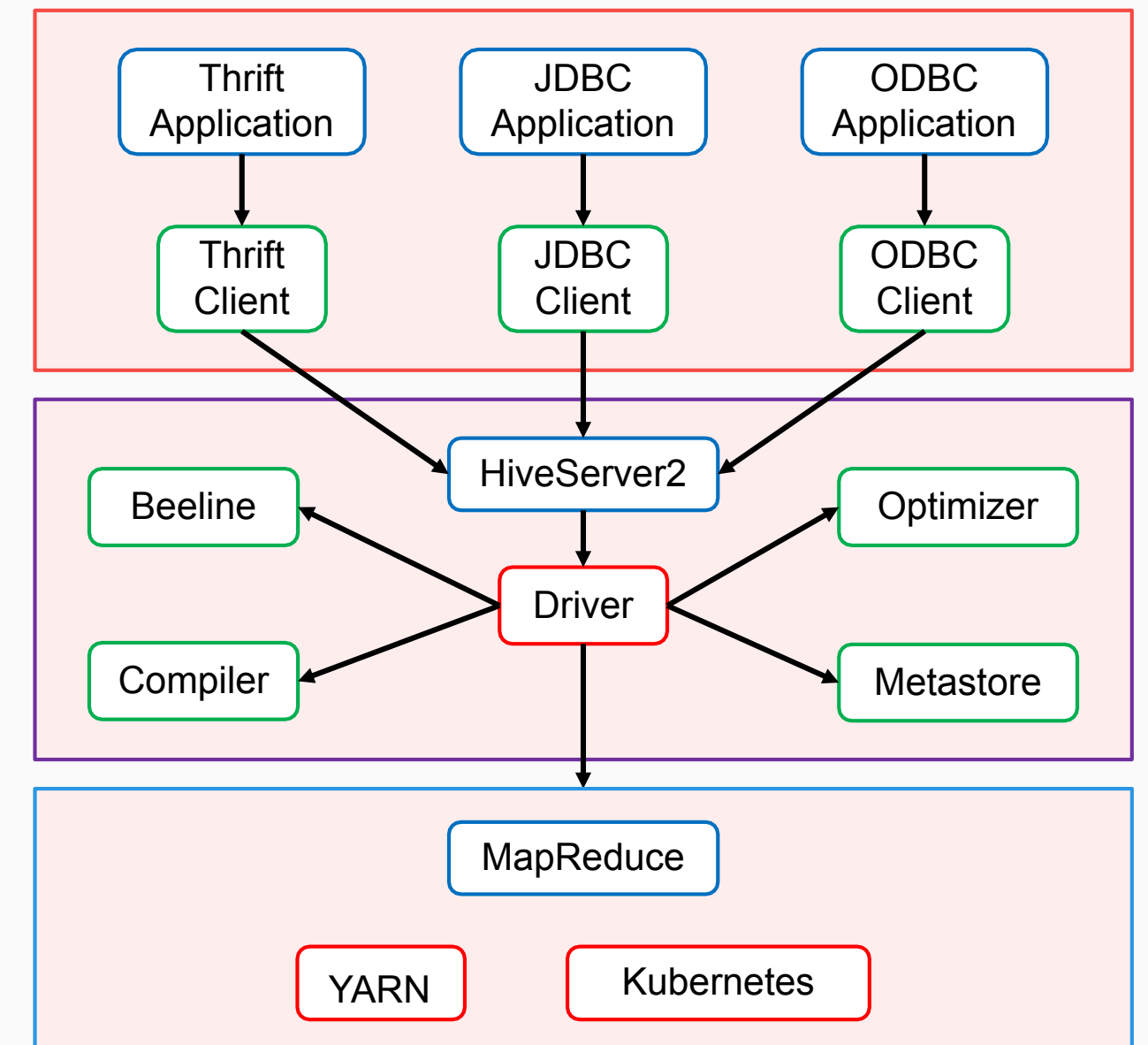
- 1 Контейнер = JVM
- 2 Резерв памяти
- 3 Резерв ядер





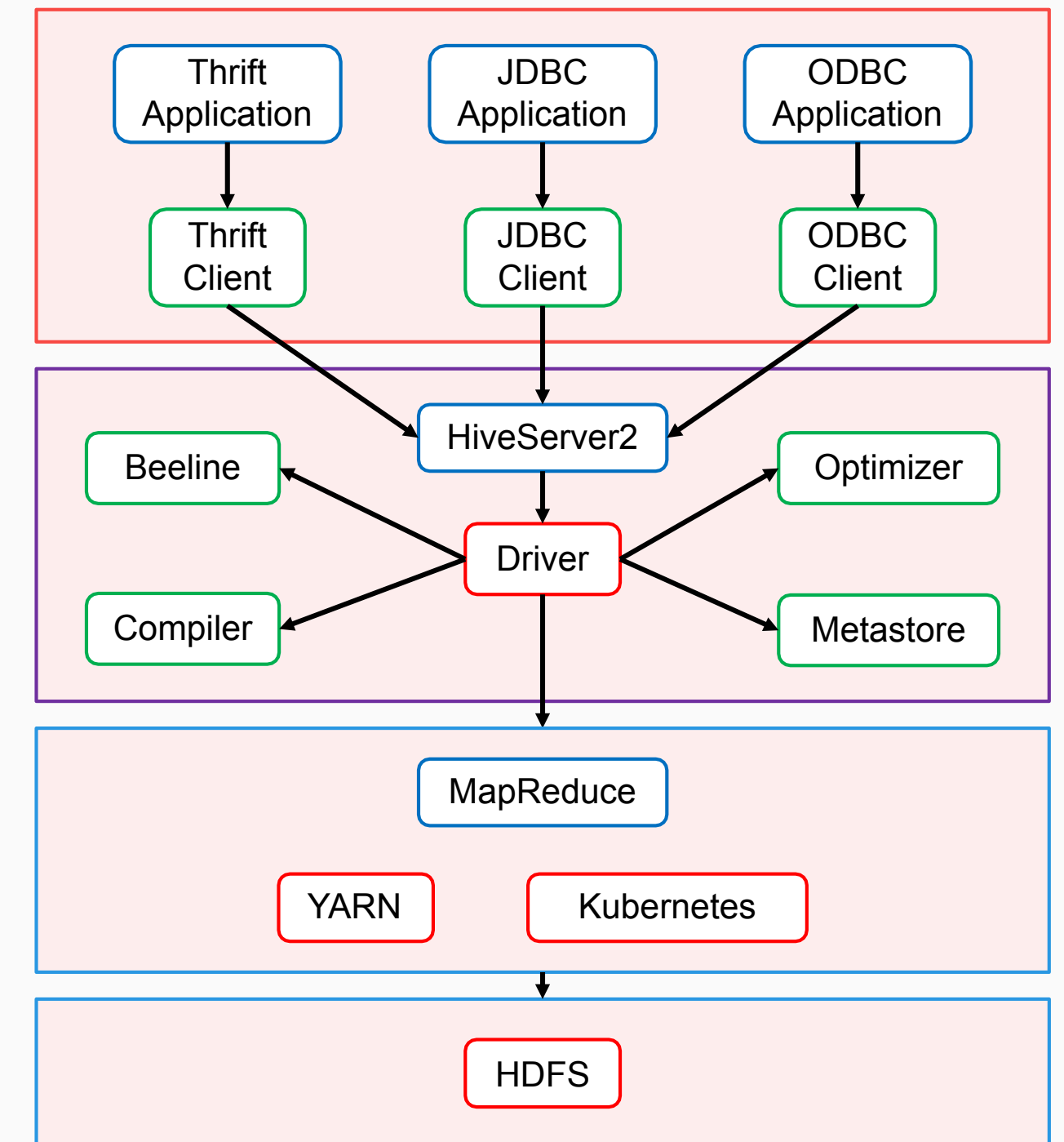
# Управление ресурсами

- 1 Контейнер = JVM
- 2 Резерв памяти
- 3 Резерв ядер
- 4 Альтернатива



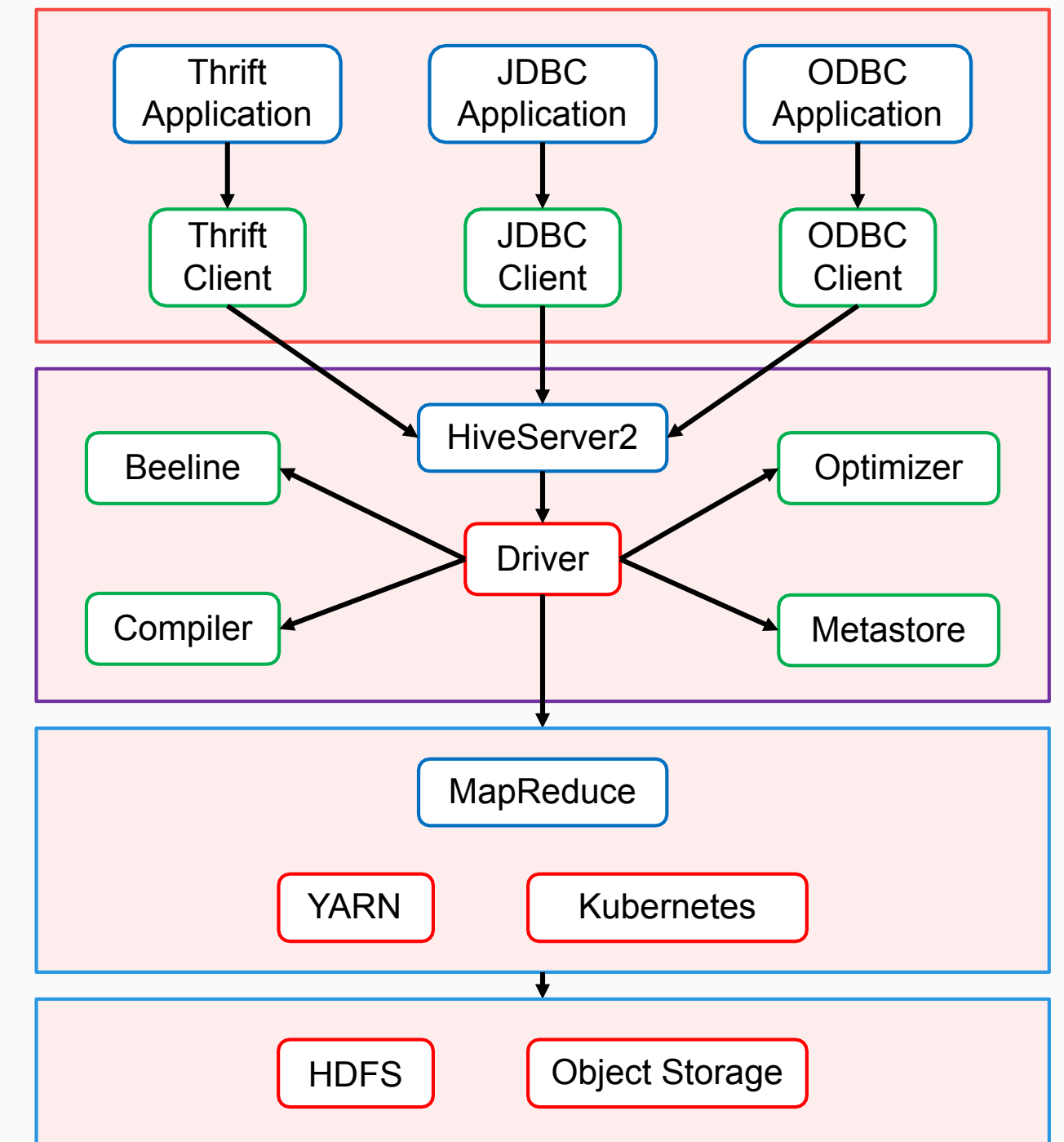
# Распределённое хранилище

- ❶ Данные в блоках
- ❷ Аллокация Java-кода

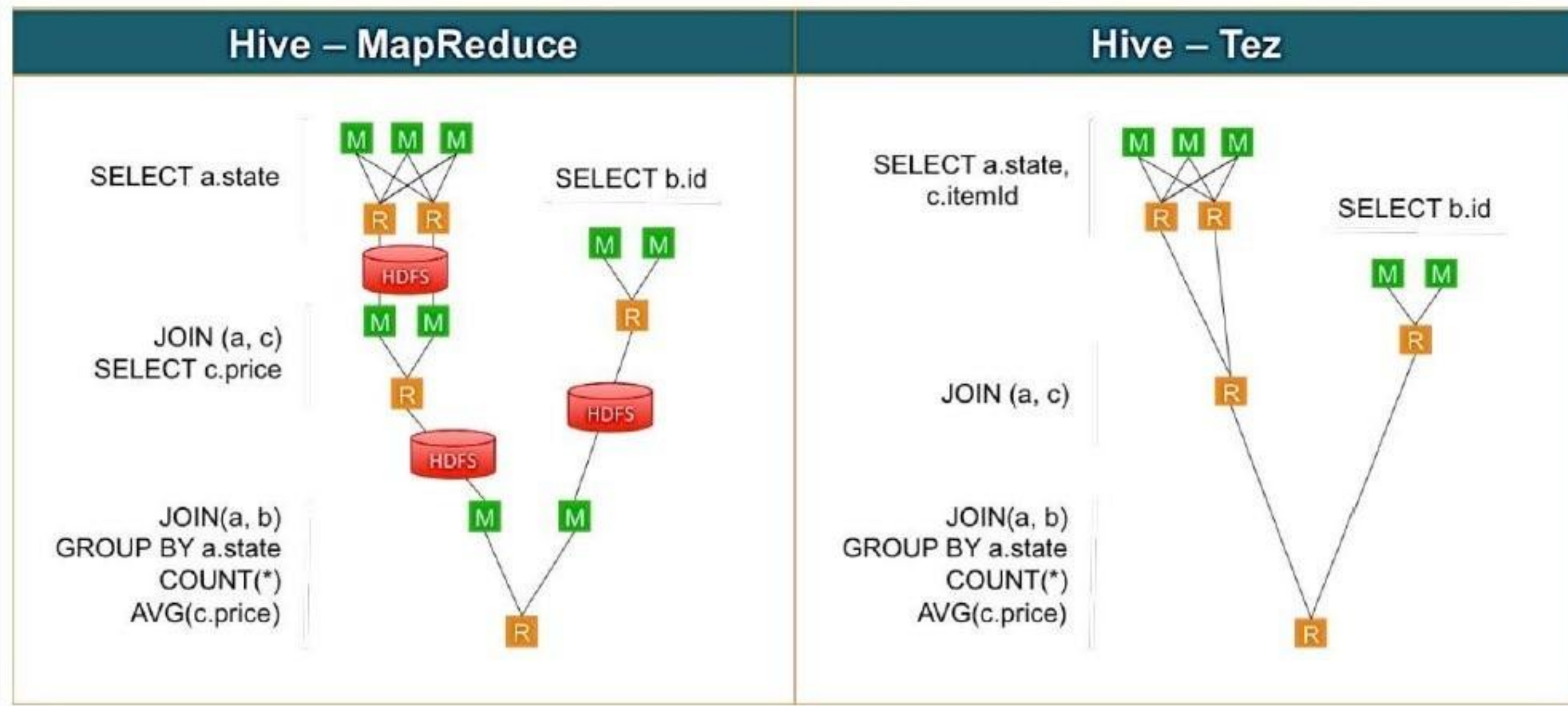


# Распределённое хранилище

- 1 Данные в блоках
- 2 Аллокация Java-кода
- 3 Альтернатива



# MapReduce vs Tez



# Apache HIVE работает вместе с...

- 1 Распределёнными хранилищами — Hadoop HDFS, Amazon S3, Minio  
Форматами данных — JSON, CSV, AVRO, Apache ORC, Parquet
- 2 SQL-движками — Presto, Hive, Spark SQL
- 3 Execution-движками — Hadoop, Tez, Apache Spark
- 4 Ресурсными менеджерами — Hadoop Yarn, Kuberne
- 5



# Окружение для занятия



# У Hive есть конфигурируемый метастор с метасторами

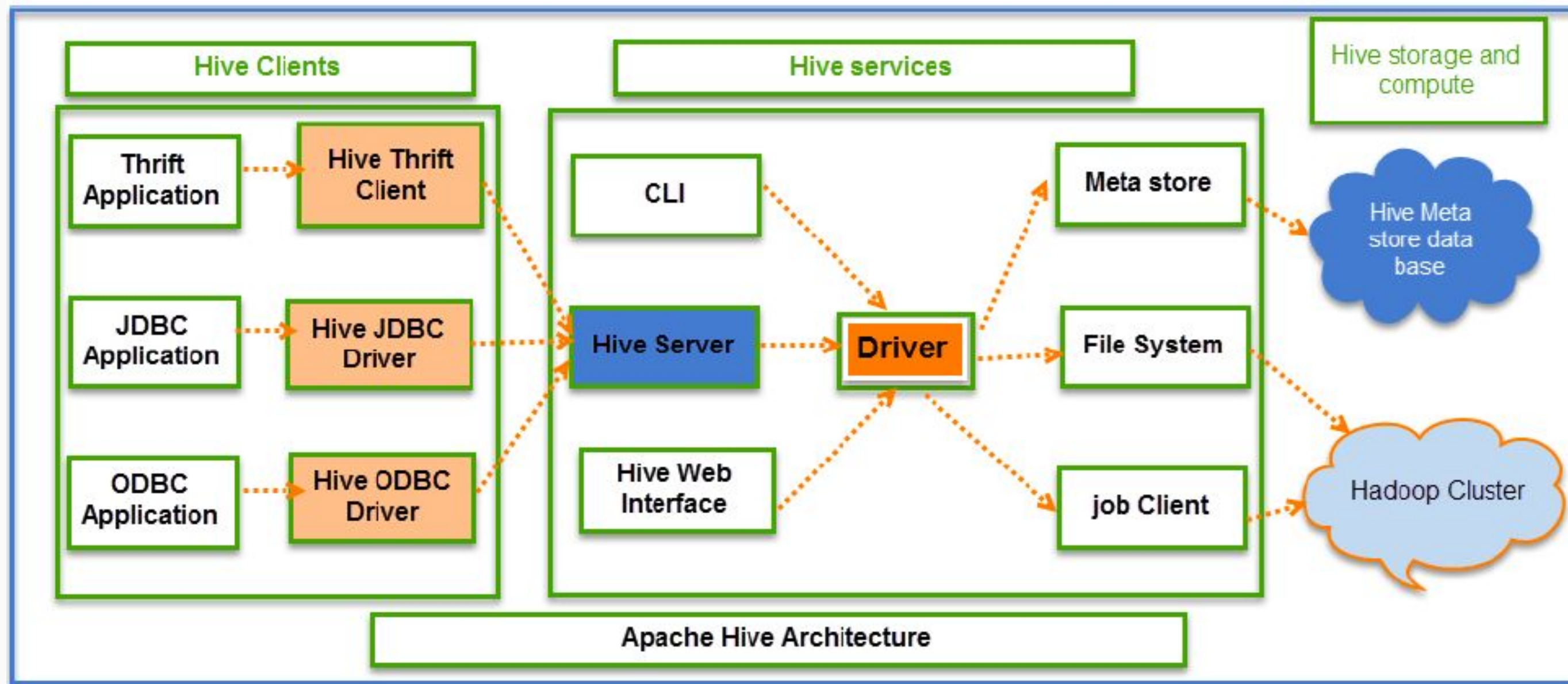


# Метаданные Hive

- 1 Hive хранит метаданные в рел. БД  
По default Derby, «легковесной» SQL DB
- 2  
Можно переключиться на другую БД
- 3



# Как это всё связано?



# Что происходит между запросом и результатом?

Шаг 1: `executeQuery`. Пользовательский интерфейс вызывает интерфейс `execute` для драйвера.

Шаг 2: `getPlan`. Драйвер принимает запрос, создаёт сеансовую обработку для запроса и передаёт запрос компилятору для создания плана выполнения.

Шаг 3: `getMetaData`. Компилятор отправляет запрос на получение метаданных в метаданное хранилище.

Шаг 4: `sendMetaData`. Метаданные отправляются компилятору из метаданного хранилища. Компилятор использует эти метаданные для проверки типов. Затем компилятор создаёт план выполнения (DAG).

# Что происходит между запросом и результатом?

Шаг 5: `sendPlan`. Затем компилятор отправляет созданный план выполнения драйверу.

Шаг 6: `executePlan`. После получения плана выполнения от компилятора драйвер отправляет план выполнения в исполнительный механизм для его выполнения.

Шаг 7: `submit job to MapReduce`. Исполнительный механизм затем отправляет эти стадии DAG соответствующим компонентам.

# Итоги

- ❶ Hive open-source SQL-движок на Java
- ❷ Транслирует SQL в MapReduce Java-код, который выполняется на Execution-движке